

Threat Detection & Incident Response using Splunk Enterprise

Security Operations Center (SOC) Laboratory

Prepared By

IBRAHIM KHALEEL

Splunk Enterprise 10.4.x | Ubuntu Server 24.04.4 LTS | Windows 11 Pro | Kali Linux | Sysmon v15.21 |
Splunk Universal Forwarder

July 2026

1. Introduction

1.1 Overview

I built this lab to get hands-on with the parts of a SOC workflow that I wanted to practice in Splunk: collecting endpoint logs, writing SPL searches, testing detections, setting up alerts, and reviewing the results in dashboards.

The setup uses Ubuntu Server for Splunk Enterprise, Windows 11 as the monitored endpoint, and Kali Linux for controlled testing. Windows Security logs, Sysmon telemetry, and Windows Firewall logs are forwarded to Splunk through the Splunk Universal Forwarder.

1.2 Project Objectives

- Deploy Splunk Enterprise as the central SIEM platform.
- Configure Windows event logging, Sysmon, and Windows Firewall logging.
- Forward telemetry with the Splunk Universal Forwarder.
- Write custom SPL searches for the detection use cases.
- Create scheduled alerts and dashboards.
- Validate the detections with controlled attack simulations.

2. Lab Architecture

2.1 System Components

Machine	Role
Ubuntu Server	Splunk Enterprise SIEM
Windows 11	Monitored endpoint
Kali Linux	Attack simulation host

2.2 Hardware and Software

Component	Specification
Processor	Intel Core i5-1135G7
Memory	16 GB RAM
Storage	512 GB SSD
Splunk	Enterprise 10.4.x
Endpoint	Windows 11 Pro
Server	Ubuntu Server 24.04.4 LTS
Attacker	Kali Linux
Telemetry	Sysmon v15.21

2.3 Network Architecture

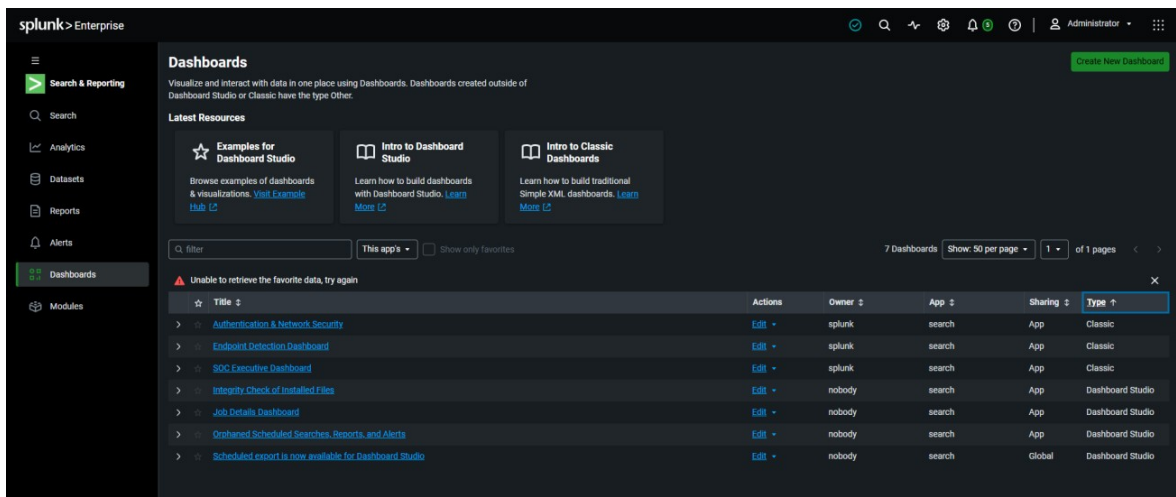


Figure 1. Laboratory network architecture

Ubuntu Server hosts Splunk Enterprise and receives telemetry from the Windows endpoint through the Universal Forwarder. Kali Linux is used to generate the controlled test activity, including PowerShell-related testing, authentication testing, and Nmap scanning.

2.4 Data Flow

1. Windows 11 generates security events.
2. Sysmon records endpoint activity such as process creation.
3. Windows Firewall records network activity.
4. The Universal Forwarder collects the configured logs.
5. Logs are forwarded to Splunk Enterprise.
6. Splunk indexes the data.
7. SPL searches analyze the events.
8. Alerts and dashboards present the results.

3. Splunk Configuration

3.1 Splunk Enterprise

Splunk Enterprise was installed on Ubuntu Server and used for log ingestion, indexing, searching, dashboards, and alerting.

3.2 Windows Endpoint Configuration

- Microsoft Sysmon v15.21
- Splunk Universal Forwarder
- Windows Firewall logging

3.3 Log Sources

Source	Purpose
Security	Authentication events
Application	Application logs
System	Operating system events
Sysmon	Process monitoring
Firewall	Network activity

3.4 Universal Forwarder

The Universal Forwarder was configured to monitor the selected Windows Event Log channels and the Windows Firewall log, then send those events to the Splunk server.

3.5 Sysmon

Sysmon gave the lab more detail around process activity. The telemetry used in the searches included Process Creation, command-line arguments, parent process information, user context, and executable paths.

4. Detection Engineering

4.1 Approach

For each use case, I first generated the activity in the lab, checked that the expected logs reached Splunk, wrote or adjusted the SPL search, and then created a scheduled alert after the search produced the expected result. The detections were also used as panels in the dashboards.

4.2 MITRE ATT&CK Coverage

Detection	Technique	Tactic
PowerShell Execution	T1059.001	Execution
Encoded PowerShell	T1059.001	Execution
Parent-Child Process	T1204	Execution
Windows Brute Force	T1110	Credential Access
TCP Port Scan	T1046	Discovery

5. Detection Use Case 1 - PowerShell Execution Detection

5.1 Objective

Identify PowerShell process creation on the monitored Windows endpoint so that an analyst can review the user, command line, and surrounding activity.

5.2 Attack Simulation

PowerShell was launched manually on the Windows endpoint using several commands. Sysmon recorded the process creation events and the Universal Forwarder sent them to Splunk.

5.3 MITRE ATT&CK

Field	Value
Technique	T1059.001
Name	PowerShell
Data Source	Sysmon
Event	EventCode 1

5.4 SPL Query

```
source="WinEventLog:Microsoft-Windows-Sysmon/Operational"
EventCode=1
| rex field=_raw "<Data Name='Image'>{<Image>[^\<]+}</Data>"
| rex field=_raw "<Data Name='CommandLine'>{<CommandLine>[^\<]*}</Data>"
| search Image="*powershell.exe"
| table _time Computer User Image CommandLine
| sort -_time
```

5.5 Query Explanation

The search filters Sysmon Process Creation events for PowerShell and extracts the image path and command line from the XML event. The final table gives the execution time, host, user, executable, and command line for investigation.

5.6 Alert

Property	Value
Alert Name	PowerShell Execution Detection
Trigger	Number of Results > 0
Severity	Medium

5.7 Screenshot

_time	Computer	User	Image	CommandLine
2026-07-27 08:09:12.715			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:09:18.561			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:08:12.688			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:08:18.572			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:07:12.717			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:07:18.959			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:06:12.786			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:06:18.954			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:05:12.638			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:05:18.957			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:04:12.715			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:04:18.942			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:03:12.701			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:03:18.931			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:02:12.681			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:02:18.927			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:01:12.691			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:01:18.924			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:00:12.693			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	
2026-07-27 08:00:18.912			C:\Program Files\SplunkUniversalForwarder\bin\splunk-powershell.exe	

Figure 2. PowerShell execution detection in Splunk

5.8 Analyst Notes

- Verify the executing user.
- Review the full command line.
- Decide whether the execution is expected.
- Correlate with other endpoint events.

6. Detection Use Case 2 - Encoded PowerShell Detection

6.1 Objective

Identify PowerShell launched with the -EncodedCommand or -enc parameter. The point of the test is to highlight command-line obfuscation that needs analyst review.

6.2 Attack Simulation

An encoded PowerShell command was executed on the Windows endpoint. Sysmon captured the process creation event and the event was forwarded to Splunk.

6.3 MITRE ATT&CK

Field	Value
Technique	T1059.001
Tactic	Execution
Data Source	Sysmon

6.4 SPL Query

```
source="WinEventLog:Microsoft-Windows-Sysmon/Operational"
EventCode=1
| rex field=_raw "<Data Name='Image'>(?!<Image>[^\<]+)</Data>"
| rex field=_raw "<Data Name='CommandLine'>(?!<CommandLine>[^\<]*)</Data>"
| where like(lower(Image), "%powershell%")
| where match(lower(CommandLine), "-enc(oddedcommand)?")
| table _time Computer User Image CommandLine
| sort - _time
```

6.5 Query Explanation

The search keeps Sysmon process-creation events where PowerShell is launched with an encoded-command switch. The command line is the main field an analyst would inspect next.

6.6 Alert

Property	Value
Alert Name	Obfuscated PowerShell Command Detection
Severity	High

6.7 Screenshot

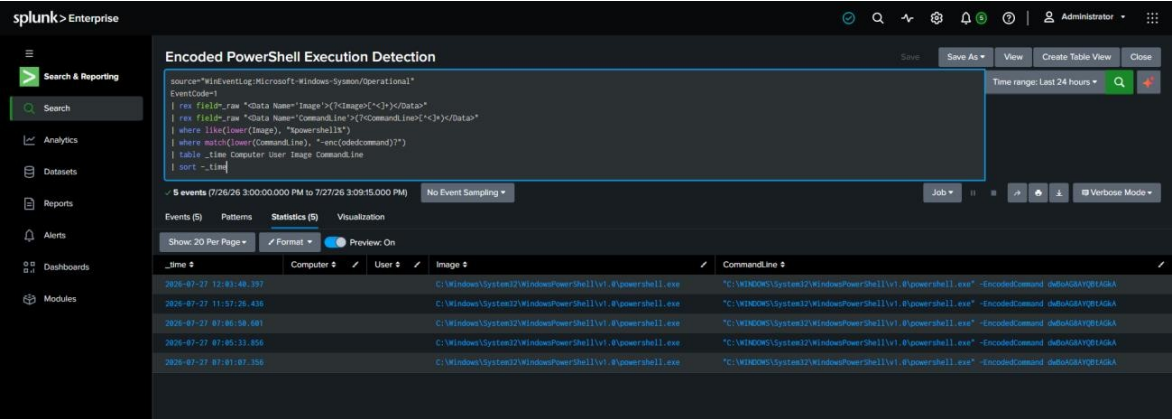


Figure 3. Encoded PowerShell detection

6.8 Analyst Notes

- Review the encoded command.
- Decode the Base64 string when required for analysis.
- Investigate the parent process.
- Check for follow-on network activity.

7. Detection Use Case 3 - Suspicious Parent-Child Process Detection

7.1 Objective

Identify unusual Windows process relationships, such as cmd.exe starting powershell.exe, using Sysmon Process Creation telemetry.

7.2 Attack Simulation

Command Prompt was used to launch PowerShell, which generated Sysmon Process Creation events containing both the parent and child process information.

7.3 MITRE ATT&CK

Field	Value
Technique	T1204
Name	User Execution
Data Source	Sysmon

7.4 SPL Query

```
source="WinEventLog:Microsoft-Windows-Sysmon/Operational"
EventCode=1
| rex field=_raw "<Data Name='Image'>(?!<Image>[^\<]+)</Data>"
| rex field=_raw "<Data Name='ParentImage'>(?!<ParentImage>[^\<]+)</Data>"
| rex field=_raw "<Data Name='CommandLine'>(?!<CommandLine>[^\<]*)</Data>"
| where ((like(lower(ParentImage), "%cmd.exe%") AND like(lower(Image), "%powershell.exe%")) OR
        (like(lower(ParentImage), "%powershell.exe%") AND like(lower(Image), "%cmd.exe%")) OR
        (like(lower(ParentImage), "%powershell.exe%") AND like(lower(Image), "%powershell.exe%")))
| table _time ParentImage Image CommandLine
| sort -_time
```

7.5 Query Explanation

The search extracts the parent process, child process, and command line from Sysmon events, then filters for the process relationships covered by the lab test.

7.6 Alert

Property	Value
Alert Name	Suspicious Parent-Child Process Detection
Severity	High

7.7 Screenshot

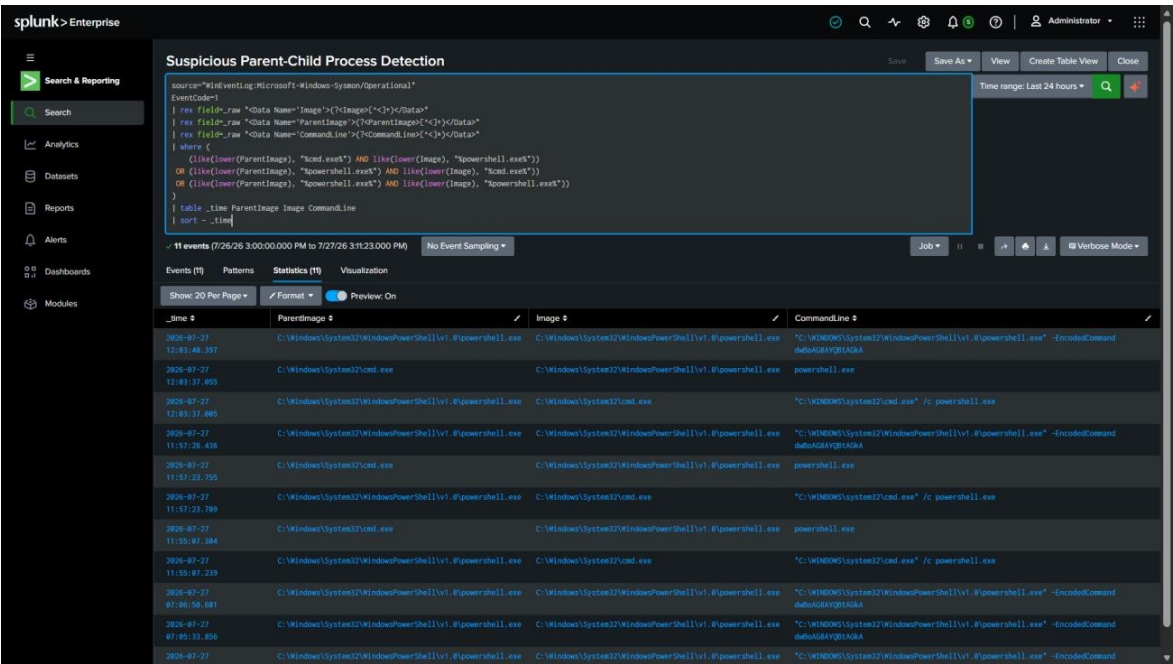


Figure 4. Parent-Child Process Detection

7.8 Analyst Notes

- Verify the parent process.
- Review the child process.
- Examine the command line.
- Correlate with authentication events.
- Check related network connections.

8. Detection Use Case 4 - Windows Authentication Brute Force Detection

8.1 Objective

Identify repeated failed Windows logons from the same source IP and provide enough context for an analyst to investigate password-guessing activity.

8.2 Attack Simulation

A controlled authentication test was performed from Kali Linux against the Windows RDP service. Multiple invalid attempts generated Windows Security Event 4625, which was forwarded to Splunk.

8.3 MITRE ATT&CK Mapping

Field	Value
Technique	T1110
Technique Name	Brute Force
Tactic	Credential Access
Log Source	Windows Security Log

8.4 SPL Query

```
index=* EventCode=4625
| eval Source_IP=coalesce(Source_Network_Address, SourceIp, IpAddress)
| stats count AS failed_attempts values(Account_Name) AS targeted_accounts by Source_IP
| where failed_attempts >= 5
| sort - failed_attempts
```

8.5 Query Explanation

The query collects failed logons, normalizes the source IP field, groups the events by source, and counts attempts. A source with five or more failed attempts is flagged for review.

8.6 Alert Configuration

Property	Value
Alert Name	Windows Authentication Brute Force Detection
Search Type	Scheduled
Trigger	Number of Results > 0
Severity	High

8.7 Expected Output

- Source IP Address
- Number of Failed Attempts
- Targeted User Accounts
- Timestamp

8.8 Screenshot

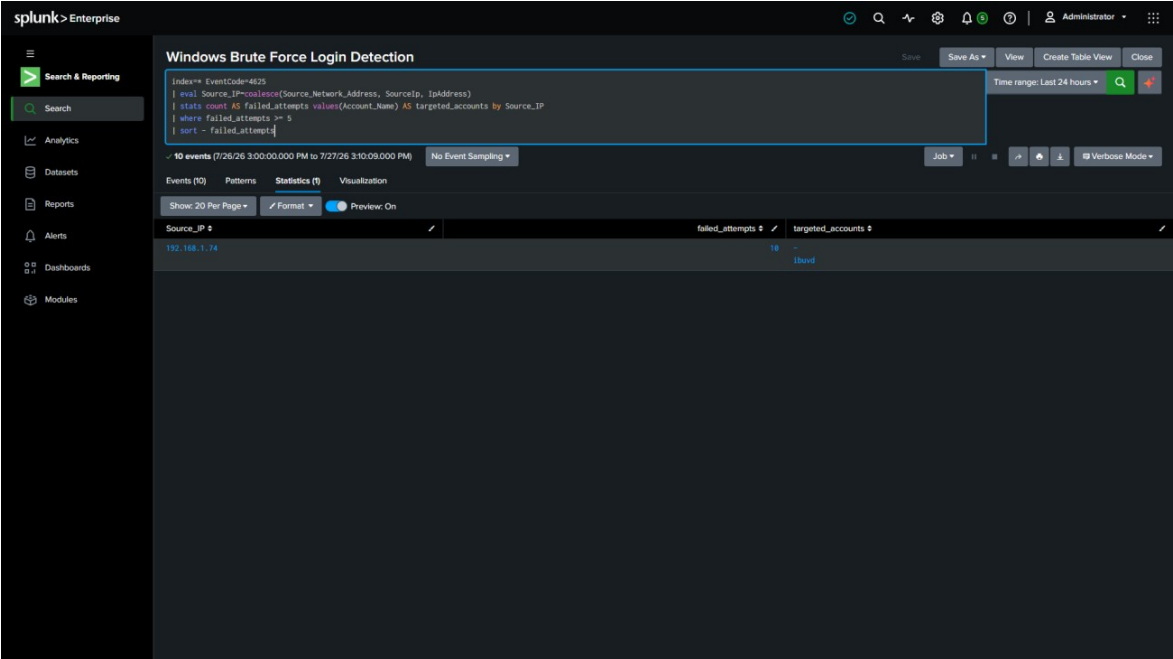


Figure 5. Windows Authentication Brute Force Detection

8.9 Analyst Investigation

- Verify whether the source is internal or external.
- Review the targeted accounts.
- Check for successful logons using EventCode 4624.
- Correlate with Sysmon and firewall events.
- Block the source only after confirming the activity is unauthorized.

9. Detection Use Case 5 - TCP Port Scan Detection

9.1 Objective

Detect TCP port scanning by analyzing Windows Firewall logs. The lab uses Nmap from Kali Linux to simulate reconnaissance against the Windows endpoint.

9.2 Attack Simulation

An Nmap TCP scan was run from Kali against the Windows endpoint. Windows Firewall recorded the incoming connection attempts in pfirewall.log, and the Universal Forwarder sent that log to Splunk.

9.3 MITRE ATT&CK Mapping

Field	Value
Technique	T1046
Technique Name	Network Service Discovery
Tactic	Discovery
Data Source	Windows Firewall Log

9.4 SPL Query

```
index=main sourcetype=pfirewall
| rex field=_raw "(?<action>ALLOW | DROP)\s+(?<protocol>\S+)\s+(?<src_ip>\S+)\s+(?<dest_ip>\S+)\s+(?<src_port>\d+|-)\s+(?<dest_port>\d+|-)"
| search protocol=TCP
| stats dc(dest_port) AS unique_ports values(dest_port) AS scanned_ports count AS total_connections by src_ip
| where unique_ports >= 3
| rename src_ip AS "Source IP"
| rename unique_ports AS "Unique Ports Scanned"
| rename scanned_ports AS "Ports"
| rename total_connections AS "Connection Attempts"
```

9.5 Query Explanation

The search extracts the network fields from the firewall log, keeps TCP activity, and counts distinct destination ports by source IP. A source reaching at least three unique ports is treated as a reconnaissance lead.

9.6 Alert Configuration

Property	Value
Alert Name	Network Reconnaissance Detection
Search Type	Scheduled
Trigger	Number of Results > 0
Severity	Medium

9.7 Expected Output

- Source IP
- Number of Unique Ports Scanned
- Scanned Ports
- Total Connection Attempts

9.8 Screenshot

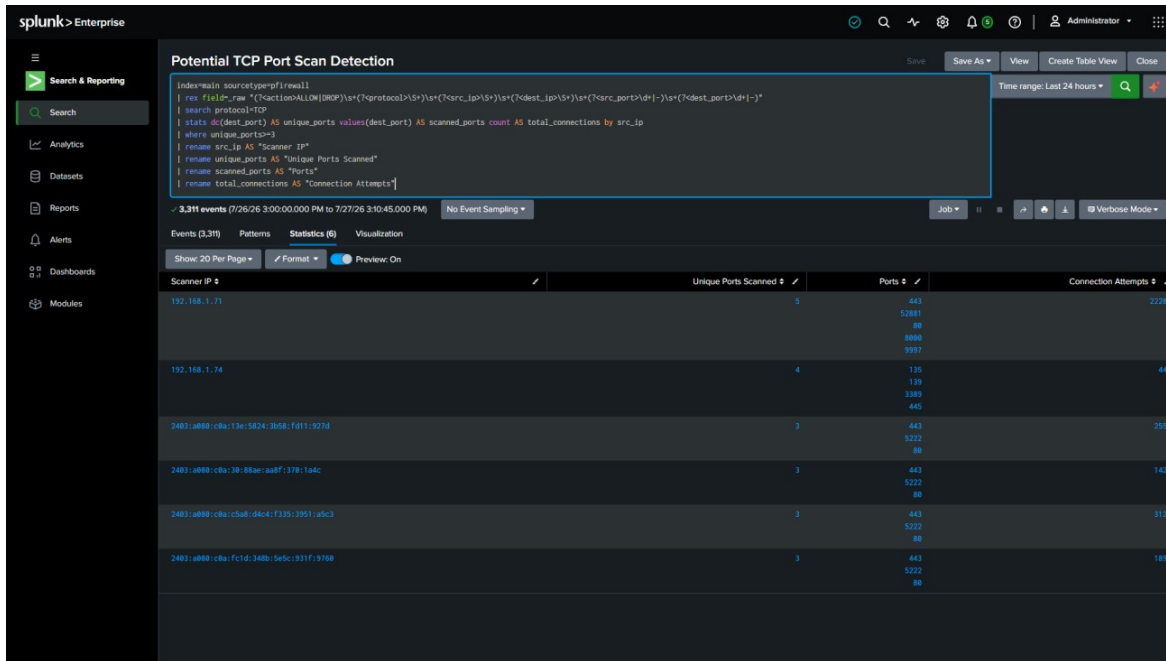


Figure 6. TCP Port Scan Detection

9.9 Analyst Investigation

- Check whether the source belongs to an approved scanner.
- Review the ports that were scanned.
- Look for subsequent exploitation attempts.
- Correlate the activity with authentication and Sysmon events.
- Monitor or block the source if the activity is unauthorized.

9.10 Alert Engineering

Once the search produced the expected result, it was saved as a scheduled alert. The flow used in the lab was:

9. Attack activity
10. Windows Security / Sysmon / Firewall logs
11. Splunk indexing
12. Scheduled SPL search
13. Detection matched
14. Alert generated
15. SOC dashboard
16. Analyst investigation

Name	Actions	Type	Next Scheduled Time	Display View	Owner	App	Alerts	Sharing	Status
Suspicious TCP Port Scan Detection Detects a source IP attempting connections to multiple TCP destination ports within the monitoring window, indicating possible reconnaissance or port scanning activity.	Edit Run View Recent	Alert	2026-07-27 15:05:00 UTC	none	spunk	search	17	App	✓ Enabled
Encoded PowerShell Execution Detection Detects PowerShell execution using EncodedCommand or other a common technique used to obfuscate malicious commands.	Edit Run View Recent	Alert	2026-07-27 15:05:00 UTC	none	spunk	search	96	App	✓ Enabled
Windows Brute Force Login Detection Detects repeated failed Windows login attempts from the same source IP, indicating a possible brute force attack.	Edit Run View Recent	Alert	2026-07-27 15:05:00 UTC	none	spunk	search	58	App	✓ Enabled
Suspicious Parent-Child Process Detection Detects suspicious parent-child process relationships involving PowerShell and Command Prompt that may indicate script execution or malicious command chaining.	Edit Run View Recent	Alert	2026-07-27 15:05:00 UTC	none	spunk	search	36	App	✓ Enabled
PowerShell Execution Alert Detects PowerShell execution using Splunk Search (SI).	Edit Run View Recent	Alert	2026-07-27 15:05:00 UTC	none	spunk	search	8	App	✓ Enabled
PowerShell Execution Detection Detects Windows PowerShell process creation using Splunk Search (SI).	Edit Run View Recent	Report	none	none	spunk	search	0	Private	✓ Enabled

Figure 7. Configured alerts in Splunk Enterprise

The five alert scenarios validated in this phase were PowerShell execution, obfuscated PowerShell, suspicious parent-child process activity, Windows authentication brute force, and TCP port scanning.

10. Dashboard Analysis

10.1 Overview

I built three dashboard views to make the detections easier to review. One gives a high-level SOC view, one focuses on endpoint process activity, and one focuses on authentication and network activity.

10.2 SOC Executive Dashboard

This view summarizes the main security activity in the lab for a quick triage pass.

- Total process events
- Detection rules triggered
- Failed logons
- Detected port scanners
- Encoded PowerShell alerts
- Recent security detections
- Event trend

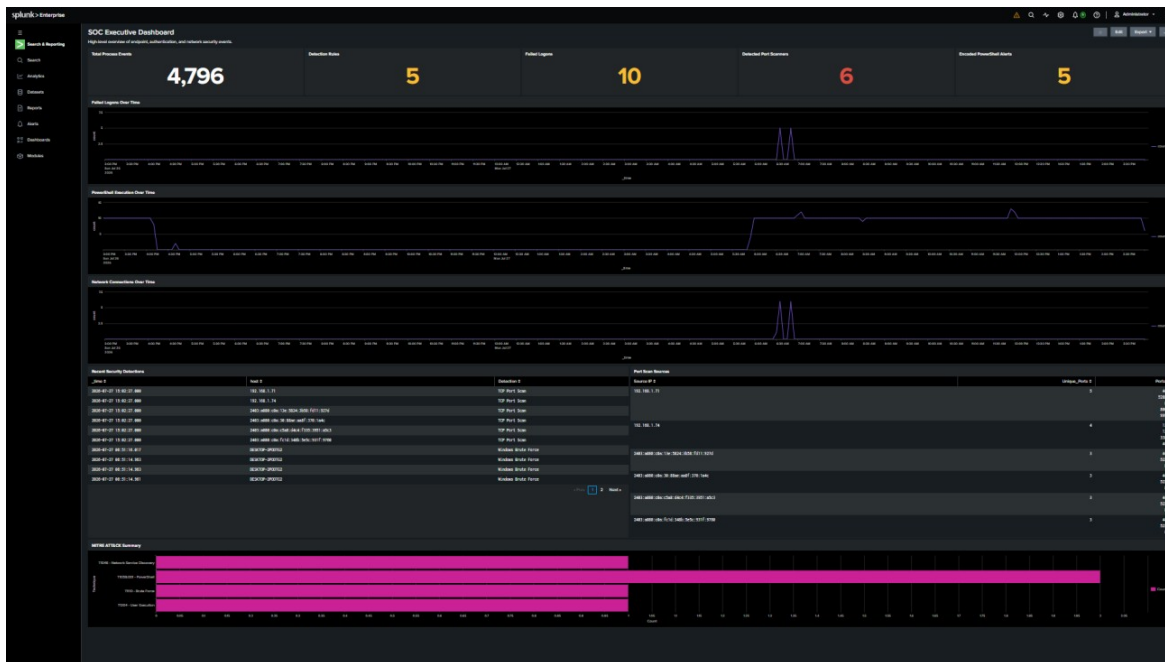


Figure 8. SOC Executive Dashboard

10.3 Endpoint Detection Dashboard

This view focuses on Sysmon-based process telemetry and is used to investigate PowerShell activity and parent-child relationships.

- PowerShell activity
- Encoded PowerShell events
- Parent-child process events
- Recent process executions
- Process relationship table

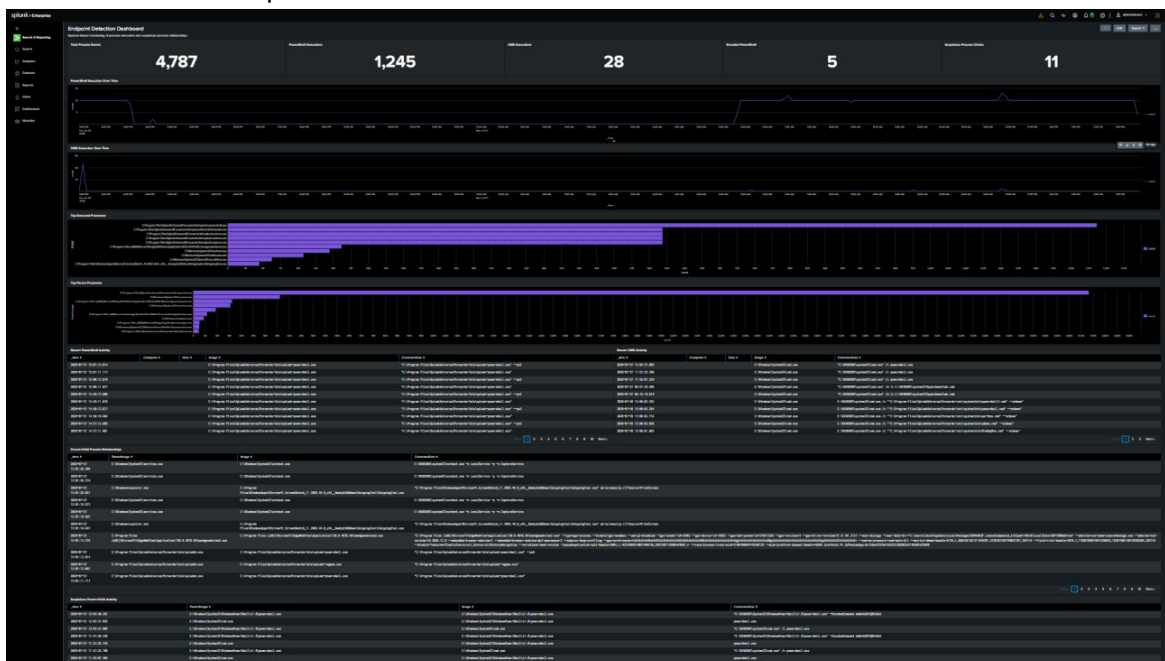


Figure 9. Endpoint Detection Dashboard

10.4 Authentication & Network Dashboard

This view brings authentication and network activity together so failed logons and port-scan activity can be reviewed in the same place.

- Failed logons
- Successful logons
- Port-scan summary
- Network connection trends
- Recent authentication events

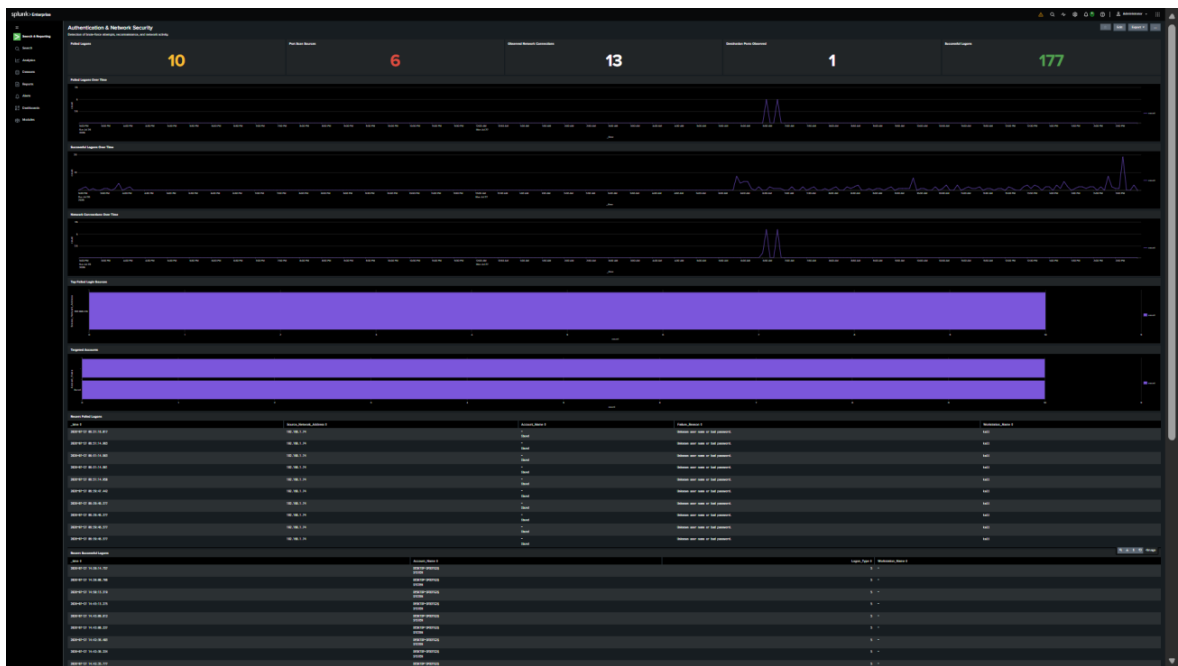


Figure 10. Authentication & Network Security Dashboard

11. Testing & Results

11.1 Validation

After the ingestion and detection configuration was in place, I repeated the controlled tests and checked whether each search returned the expected data and whether the corresponding alert was generated.

Detection	Test	Alert	Status
PowerShell Execution	Manual PowerShell	Yes	Successful
Encoded PowerShell	Base64 command	Yes	Successful
Parent-Child Process	cmd.exe -> powershell.exe	Yes	Successful
Windows Brute Force	Hydra RDP test	Yes	Successful
TCP Port Scan	Nmap scan	Yes	Successful

11.2 Results Summary

- Centralized Windows telemetry was ingested into Splunk.
- Sysmon provided process-level endpoint visibility.
- Windows Firewall logs were parsed for network activity.
- Custom SPL searches were used for the five detection scenarios.
- Scheduled alerts were created and validated.
- The detections were represented in dashboards.
- MITRE ATT&CK techniques were associated with the observed behaviors.

12. Problems Faced & Troubleshooting

Problem	What I did
Sysmon fields were not extracted automatically	Used rex to extract fields such as Image, ParentImage, and CommandLine.
Universal Forwarder connection issues	Checked network connectivity, the receiving port, and firewall settings.
Windows Firewall logs were not parsing correctly	Built custom field extraction with rex.
EventID/EventCode confusion	Used EventCode consistently in the Splunk searches.
Detection results were missing	Checked ingestion, corrected the SPL, and reran the controlled test.

These issues were useful because they forced me to check the whole pipeline instead of assuming that an empty search meant the detection logic was wrong.

13. Conclusion

This lab gave me a practical Splunk environment in which I could work through log collection, SPL development, detection validation, alerting, dashboards, and basic SOC investigation. The five tested scenarios covered PowerShell execution, encoded PowerShell, suspicious process relationships, failed authentication, and TCP port scanning.

The most useful part of the work was the troubleshooting. When a detection did not appear, I had to trace the problem from the source log through the Universal Forwarder and Splunk search before changing the detection itself.

14. Future Improvements

- Add VirusTotal integration for file-hash reputation checks.
- Add Active Directory log monitoring.
- Add network sensors such as Suricata or Zeek.
- Bring in external threat-intelligence feeds.
- Experiment with UEBA-style detections.
- Test automated response using a SOAR platform.
- Add ransomware behavior detections.
- Expand the lab to multiple Windows endpoints.

15. References

17. Splunk Enterprise Documentation
18. Splunk Search Processing Language (SPL) Reference
19. Microsoft Sysmon Documentation
20. Microsoft Windows Security Auditing Documentation
21. MITRE ATT&CK Framework
22. Nmap Reference Guide
23. THC Hydra Documentation

Appendix A - Detection Summary

Detection	Log Source	MITRE Technique	Alert
PowerShell Execution	Sysmon	T1059.001	Yes
Obfuscated PowerShell	Sysmon	T1059.001	Yes
Parent-Child Process	Sysmon	T1204	Yes
Windows Brute Force	Security Log	T1110	Yes
TCP Port Scan	Windows Firewall	T1046	Yes